

Proxylab(1)

实验环境

1. VScode
2. Ubuntu 20.04.6 LTS

实验要求

1. 参照实验手册完成实验第一部分(基本顺序代理)
2. 代码逻辑清晰，必要处有注释实验报告需要有详细的分析过程和说明

实验内容

代码

```
racalia202212063020@THESEI  ×  +  ~
#include <stdio.h>
#include "csapp.h"
#define MAX_CACHE_SIZE 1049000
#define MAX_OBJECT_SIZE 102400
//学号 202212063020
static const char *user_agent_hdr = "User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:10.0.3) Gecko/20120305 Firefox/10.0.3\r\n";
void doit(int clientfd);
int parse_uri(char *uri, char *hostname, char *path, char *port, char *request_head);
void read_requesthdrs(rio_t *rp, int fd);
void return_content(int serverfd, int clientfd);
int main(int argc, char **argv)
{
    int listenfd, connfd;
    char hostname[MAXLINE], port[MAXLINE];
    socklen_t clientlen;
    struct sockaddr_storage clientaddr;
    if(argc != 2)
    {
        fprintf(stderr, "usage: %s <port>\n", argv[0]);
        exit(1);
    }
    /* 代理创建一个监听描述符, 准备好接收连接请求 */
    listenfd = Open_listenfd(argv[1]);
    while(1)
    {
        clientlen = sizeof(clientaddr);
        /* 等待来着客户端的连接请求到侦听描述符 listenfd,
        然后在 addr 中填写客户端的套接字地址, 并返回一个已连接描述符 */
        connfd = Accept(listenfd, (SA *)&clientaddr, &clientlen);
        /* 将套接字地址结构 clientaddr 转化成对应的主机和服务名字符串,
        并将他们复制到 hostname 和 port 缓冲区 */
        Getnameinfo((SA *)&clientaddr, clientlen, hostname, MAXLINE, port, MAXLINE, 0);
        printf("Accepted connection from (%s, %s)\n", hostname, port);
        doit(connfd);
        Close(connfd);
    }
}
/* 处理客户端 HTTP 事务 */
void doit(int clientfd)
{
    char buf[MAXLINE], method[MAXLINE], uri[MAXLINE], version[MAXLINE];
    char hostname[MAXLINE], path[MAXLINE], port[MAXLINE], request_head[MAXLINE];
    int serverfd;
-- INSERT --
```



```
rio_t rio;
/* 读取请求行和请求头 */
Rio_readinitb(&rio,clientfd);
Rio_readlineb(&rio,buf,MAXLINE);
sscanf(buf,"%s %s %s",method,uri,version);
if(strcasecmp(method,"GET"))
{
    printf("Not implemented");
    return;
}
/* 解析 uri 取 hostname 和 path 和 port。生成 request_head */
parse_uri(uri,hostname,path,port,request_head);
/* 建立与服务器的连接 */
serverfd = Open_clientfd(hostname,port);
/* 将请求头传递给服务器 */
Rio_writen(serverfd,request_head,strlen(request_head));
read_requesthdrs(&rio,serverfd);
/* 将服务端读取的数据返回给客户端 */
return_content(serverfd,clientfd);
}
/* 解析 uri 中的 hostname 和 path 和 port。并生成 request_head
* uri 例子: http://www.cmu.edu:8080/hub/index.html
* hostname:www.cmu.edu
* path:/hub/index.html
* port:8080
*/
int parse_uri(char *uri,char *hostname,char *path,char *port,char *request_head)
{
    sprintf(port,"80"); //默认值
    char *end,*bp;
    char *tail = uri+strlen(uri); //uri 的最后一个字符, 不是'\0'。
    char *bg = strstr(uri,"//");
    bg = (bg!=NULL ? bg+2 : uri); //取 hostname 的开头。
    end = bg;
    //取 hostname 的结尾。
    while(*end != '/' && *end != ':') end++;
    strncpy(hostname,bg,end-bg);
    bp = end + 1; //取 port 的开头
    if(*end == ':')
    { //==':'说明 uri 中有 port
        end++;
        bp = strstr(bg,"/"); //取 port 的结尾
        strncpy(port,end,bp-end);
        end = bp; //取 uri 的开头
    }
    strncpy(path,end,(int)(tail-end)+1);
}
-- INSERT --
```

```
}
strncpy(path,end,(int)(tail-end)+1);
/* 请求的开头行: GET /hub/index.html HTTP/1.0。 */
sprintf(request_head,"GET %s HTTP/1.0\r\nHost: %s\r\n",path,hostname);
return 1;
}
/*
* 读取 HTTP 请求头
* Host,User-Agent,Connection 和 Proxy-Connection 用指定的的
* 保留其他的头(header)
*/
void read_requesthdrs(rio_t *rp,int fd)
{
    char buf[MAXLINE];
    sprintf(buf,"%s", user_agent_hdr);
    Rio_writen(fd, buf, strlen(buf));
    sprintf(buf,"Connection: close\r\n");
    Rio_writen(fd, buf, strlen(buf));
    sprintf(buf,"Proxy-Connection: close\r\n");
    Rio_writen(fd, buf, strlen(buf));
    /* 保留其他的头(header) */
    for(Rio_readlineb(rp,buf,MAXLINE);strcmp(buf,"\r\n");Rio_readlineb(rp,buf,MAXLINE))
    {
        if(strncmp("Host",buf,4) == 0 || strncmp("User-Agent",buf,10) == 0
           || strncmp("Connection",buf,10) == 0 || strncmp("Proxy-Connection",buf,16) == 0)
            continue;
        printf("%s",buf);
        Rio_writen(fd,buf,strlen(buf));
    }
    Rio_writen(fd,buf,strlen(buf));
    return;
}
/*
* 将服务端读取的数据返回给客户端
*/
void return_content(int serverfd, int clientfd)
{
    size_t n;
    char buf[MAXLINE];
    rio_t srrio;
    Rio_readinitb(&srrio,serverfd);
    while((n = Rio_readlineb(&srrio,buf,MAXLINE)) != 0)
    {
        Rio_writen(clientfd,buf,n);
    }
}
-- INSERT --
```

截图

```
*** Basic ***
Starting tiny on 10036
Starting proxy on 9734
1: home.html
  Fetching ./tiny/home.html into ../.proxy using the proxy
  Fetching ./tiny/home.html into ../.noproxy directly from Tiny
  Comparing the two files
  Success: Files are identical.
2: csapp.c
  Fetching ./tiny/csapp.c into ../.proxy using the proxy
  Fetching ./tiny/csapp.c into ../.noproxy directly from Tiny
  Comparing the two files
  Success: Files are identical.
3: tiny.c
  Fetching ./tiny/tiny.c into ../.proxy using the proxy
  Fetching ./tiny/tiny.c into ../.noproxy directly from Tiny
  Comparing the two files
  Success: Files are identical.
4: godzilla.jpg
  Fetching ./tiny/godzilla.jpg into ../.proxy using the proxy
  Fetching ./tiny/godzilla.jpg into ../.noproxy directly from Tiny
  Comparing the two files
  Success: Files are identical.
5: tiny
  Fetching ./tiny/tiny into ../.proxy using the proxy
  Fetching ./tiny/tiny into ../.noproxy directly from Tiny
  Comparing the two files
  Success: Files are identical.
Killing tiny and proxy
basicScore: 40/40
```